

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Insection dan Bubble sort”

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

KATA PENGANTAR

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Merge sort bekerja dengan membagi array menjadi dua bagian secara rekursif hingga tersisa satu elemen, lalu menggabungkannya kembali dalam urutan yang benar — stabil dan konsisten di $O(n \log n)$ tapi butuh memori ekstra. Quick sort memilih sebuah pivot lalu mempartisi array sehingga elemen lebih kecil ada di kiri dan lebih besar di kanan, kemudian rekursif pada kedua sisi — rata-rata $O(n \log n)$ dan in-place, tapi bisa degradasi ke $O(n^2)$ jika pemilihan pivot buruk. Shell sort adalah pengembangan insertion sort yang menyortir elemen-elemen yang berjarak gap tertentu terlebih dahulu, lalu memperkecil gap secara bertahap hingga 1 — tidak seefisien dua yang lain secara teoritis, tapi sederhana, in-place, dan performanya cukup baik untuk data berukuran sedang.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Merge sort
2. Memahami konsep dasar Shell sort
3. Memahami konsep dasar Quick sort

BAB 2 PEMBAHASAN

3.1 Program Merge Sort

```

2  package pekan8_2511532021;
3
4  public class mergeSort_2511532021 {
5
6      void merge(int arr_2021[], int l_2021, int m_2021, int r_2021) {
7
8          int n1_2021 = m_2021 - l_2021 + 1;
9          int n2_2021 = r_2021 - m_2021;
10
11          int L_2021[] = new int[n1_2021];
12          int R_2021[] = new int[n2_2021];
13
14
15          for (int i_2021 = 0; i_2021 < n1_2021; i_2021++) {
16              L_2021[i_2021] = arr_2021[l_2021 + i_2021];
17          }
18
19
20          for (int j_2021 = 0; j_2021 < n2_2021; j_2021++) {
21              R_2021[j_2021] = arr_2021[m_2021 + 1 + j_2021];
22          }
23
24          int i_2021 = 0;
25          int j_2021 = 0;
26          int k_2021 = l_2021;
27
28
29          while (i_2021 < n1_2021 && j_2021 < n2_2021) {
30
31              if (L_2021[i_2021] <= R_2021[j_2021]) {
32                  arr_2021[k_2021] = L_2021[i_2021];
33                  i_2021++;
34              } else {
35                  arr_2021[k_2021] = R_2021[j_2021];
36                  j_2021++;
37              }
38
39              k_2021++;
40          }
41
42
43          while (i_2021 < n1_2021) {
44              arr_2021[k_2021] = L_2021[i_2021];
45              i_2021++;
46              k_2021++;
47          }
48
49
50          while (j_2021 < n2_2021) {
51              arr_2021[k_2021] = R_2021[j_2021];
52              j_2021++;
53              k_2021++;
54          }
55      }
56
57      void sort(int arr_2021[], int l_2021, int r_2021) {
58
59          if (l_2021 < r_2021) {
60
61              int m_2021 = (l_2021 + r_2021) / 2;
62
63              sort(arr_2021, l_2021, m_2021);
64              sort(arr_2021, m_2021 + 1, r_2021);
65
66              merge(arr_2021, l_2021, m_2021, r_2021);
67          }

```

```

68     }
69
70     public static void printArray_2021(int[] arr_2021) {
71         for (int i_2021 : arr_2021) {
72             System.out.print(i_2021 + " ");
73         }
74
75         System.out.println();
76     }
77
78     public static void main(String[] args) {
79
80         int[] arr_2021 = {12, 11, 13, 5, 6, 7};
81
82         System.out.print("Sebelum (Merge Sort): ");
83         printArray_2021(arr_2021);
84
85         mergeSort_2511532021 ob_2021 = new mergeSort_2511532021();
86
87         ob_2021.sort(arr_2021, 0, arr_2021.length - 1);
88
89         System.out.print("Sesudah (Merge Sort): ");
90         printArray_2021(arr_2021);
91     }
92 }
93
94

```

Program ini mengimplementasikan Merge Sort dengan dua method utama: `sort()` yang secara rekursif membagi array menjadi dua bagian hingga tersisa satu elemen, dan `merge()` yang menggabungkan kembali dua sub-array sementara (L dan R) dengan membandingkan elemen satu per satu sehingga hasilnya terurut ascending — lalu di `main()` array {12,11,13,5,6,7} dicetak sebelum dan sesudah proses sorting untuk memperlihatkan hasilnya.

1.1 Program Shell Sort

```

95 package pekan8_2511532021;
96
97 public class Shellsort_2511532021 {
98     public static void shellSort_2021(int[] A_2021) {
99         int n_2021 = A_2021.length;
100         int gap_2021 = n_2021 / 2;
101         while (gap_2021 > 0) {
102             for (int i_2021 = gap_2021; i_2021 < n_2021 ; i_2021++) {
103                 int temp_2021 = A_2021[i_2021];
104                 int j_2021 = i_2021;
105                 while ( j_2021 >= gap_2021 && A_2021[j_2021 - gap_2021] >
temp_2021 ) {
106                     A_2021[j_2021] = A_2021[j_2021 - gap_2021];
107                     j_2021 = j_2021 - gap_2021;
108                 }
109                 A_2021[j_2021] = temp_2021;
110             }
111             gap_2021 = gap_2021 / 2 ;
112         }
113     }
114
115     public static void printArray_2021(int[] arr_2021) {
116         for (int i_2021 : arr_2021) System.out.print(i_2021 + " ");
117         System.out.println();
118     }
119
120     public static void main(String[] args) {
121         int[] data_2021 = {3,10,4,6,8,9,7,2,1,5};
122         System.out.print("Sebelum ");
123         printArray_2021(data_2021);

```

```

124         shellSort_2021(data_2021);
125         System.out.print("Sesudah (Shell Sort): ");
126         printArray_2021(data_2021);
127     }
128
129 }
130

```

Program ini mengimplementasikan Shell Sort di mana `shellSort_2021()` bekerja dengan gap awal $n/2$ yang terus dihalving setiap iterasi — pada setiap gap, elemen-elemen yang berjarak gap dibandingkan dan digeser layaknya insertion sort, sehingga array semakin "hampir terurut" sebelum gap mencapai 1 dan melakukan insertion sort final — lalu di `main()` array {3,10,4,6,8,9,7,2,1,5} dicetak sebelum dan sesudah sorting untuk memperlihatkan hasilnya.

1.1 Program Quick Sort

```

131 package pekan8_2511532021;
132
133 public class Quicksort_2511532021 {
134     static void swap_2021(int[] arr_2021, int i_2021, int j_2021) {
135         int temp_2021 = arr_2021[i_2021];
136         arr_2021[i_2021] = arr_2021[j_2021];
137         arr_2021[j_2021] = temp_2021;
138     }
139
140     static void medianOfThree_2021(int [] arr_2021, int low_2021, int high_2021) {
141         int mid_2021 = low_2021 + (high_2021 - low_2021) / 2;
142
143         if(arr_2021[low_2021] > arr_2021[mid_2021]) {
144             swap_2021(arr_2021, low_2021, mid_2021);
145         }
146
147         if(arr_2021[low_2021] > arr_2021[high_2021]) {
148             swap_2021(arr_2021, low_2021, high_2021);
149         }
150         if(arr_2021[mid_2021] > arr_2021[high_2021]) {
151             swap_2021(arr_2021, mid_2021, high_2021);
152         }
153
154         swap_2021(arr_2021, mid_2021, high_2021);
155     }
156
157     static int partition_2021(int[] arr_2021, int low_2021, int high_2021) {
158         medianOfThree_2021(arr_2021, low_2021, high_2021);
159
160
161         int pivot_2021 = arr_2021[high_2021];
162         int i_2021 = (low_2021 - 1);
163
164         for (int j_2021 = low_2021 ; j_2021 <= high_2021 -1 ; j_2021++ ) {
165             if(arr_2021[j_2021] < pivot_2021) {
166                 i_2021++;
167                 swap_2021(arr_2021, i_2021, j_2021);
168             }
169         }
170         swap_2021(arr_2021, i_2021 + 1, high_2021);
171         return (i_2021 + 1);
172     }
173
174     static void quickSort_2021(int[] arr_2021, int low_2021, int high_2021) {
175         if(low_2021 < high_2021) {
176             int pi_2021 = partition_2021(arr_2021, low_2021, high_2021);
177             quickSort_2021(arr_2021, low_2021, pi_2021 - 1);
178             quickSort_2021(arr_2021, pi_2021 + 1, high_2021);
179         }
180     }
181 }

```

```

180     }
181
182     public static void printArray_2021(int[] arr_2021) {
183         for (int i_2021 : arr_2021) System.out.print(i_2021 + " ");
184         System.out.println();
185     }
186
187     public static void main(String[] args) {
188         int[] data_2021 = {10,7,8,9,1,5};
189         int N_2021 = data_2021.length;
190         System.out.print("Sebelum ");
191         printArray_2021(data_2021);
192         quickSort_2021(data_2021, 0 , N_2021-1);
193         System.out.print("Sesudah (Shell Sort): ");
194         printArray_2021(data_2021);
195     }
196
197 }
198

```

Program ini mengimplementasikan Quick Sort dengan median-of-three pivot. `medianOfThree_2021()` memilih pivot terbaik dari tiga elemen (low, mid, high) dengan mengurutkannya lalu menempatkan median di posisi high sebagai pivot. `partition_2021()` memanggil median-of-three lalu mempartisi array — elemen lebih kecil dari pivot dikumpulkan ke kiri, lebih besar ke kanan, lalu pivot ditempatkan di posisi finalnya dan indexnya dikembalikan. `quickSort_2021()` secara rekursif memanggil dirinya pada sisi kiri dan kanan pivot hingga sub-array berukuran 1 — lalu di `main()` array {10,7,8,9,1,5} diurutkan dan dicetak sebelum dan sesudah sorting.

1.1 Program BubbleSortGUI

```

199 package pekan8_2511532021;
200
201 import java.awt.BorderLayout;
202 import java.awt.Color;
203 import java.awt.EventQueue;
204 import java.awt.FlowLayout;
205 import java.awt.Font;
206 import java.lang.reflect.Array;
207 import java.awt.Dimension;
208
209 import javax.swing.border.EmptyBorder;
210 import javax.swing.*;
211
212
213 public class BubbleSortGUI_2511532021 extends JFrame {
214
215     private static final long serialVersionUID_2021 = 1L;
216     private int[] array_2021;
217     private JLabel[] labelArray_2021;
218     private JButton stepButton_2021, resetButton_2021, setButton_2021;
219     private JTextField inputField_2021;
220     private JPanel panelArray_2021;
221     private JTextArea stepArea_2021;
222
223     private int i_2021 = 1, j_2021;
224     private boolean sorting_2021 = false;
225     private int stepCount_2021 = 1;
226
227     private JPanel contentPane_2021;
228
229     public BubbleSortGUI_2511532021() {
230         setTitle("Insertion Sort Langkah per Langkah");
231         setSize(750, 400);
232         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
233         setLocationRelativeTo(null);
234         setLayout(new BorderLayout());
235
236         JPanel inputPanel_2021 = new JPanel(new FlowLayout());
237         inputField_2021 = new JTextField(30);
238         setButton_2021 = new JButton("Set Array");
239         inputPanel_2021.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
240         inputPanel_2021.add(inputField_2021);
241         inputPanel_2021.add(setButton_2021);
242
243         panelArray_2021 = new JPanel();
244         panelArray_2021.setLayout(new FlowLayout());
245
246         JPanel controlPanel_2021 = new JPanel();
247         stepButton_2021 = new JButton("Langkah Selanjutnya");
248         resetButton_2021 = new JButton("Reset");
249         stepButton_2021.setEnabled(false);
250         controlPanel_2021.add(stepButton_2021);
251         controlPanel_2021.add(resetButton_2021);
252
253
254         stepArea_2021 = new JTextArea(8,60);
255         stepArea_2021.setEditable(false);
256         stepArea_2021.setFont(new Font("Monospaced", Font.PLAIN, 14));
257         JScrollPane scrollPane_2021 = new JScrollPane(stepArea_2021);
258
259         add(inputPanel_2021, BorderLayout.NORTH);
260         add(panelArray_2021, BorderLayout.CENTER);
261         add(controlPanel_2021, BorderLayout.SOUTH);
262         add(scrollPane_2021, BorderLayout.EAST);
263
264         setButton_2021.addActionListener(e -> setArrayFromInput());
265         stepButton_2021.addActionListener(e -> performStep());
266         resetButton_2021.addActionListener(e -> reset());
267     }
268

```

```

269 private void setArrayFromInput() {
270     String text_2021 = inputField_2021.getText().trim();
271     if (text_2021.isEmpty())return;
272     String[] parts_2021 = text_2021.split(",");
273     array_2021 = new int[parts_2021.length];
274
275     try {
276         for (int k_2021 = 0 ; k_2021 < parts_2021.length; k_2021++) {
277             array_2021[k_2021] =Integer.parseInt(parts_2021[k_2021].trim());
278         }
279     }catch (NumberFormatException e) {
280         JOptionPane.showMessageDialog(this, "masukkan angka" + "yang dipisahkan
koma!", "Error", JOptionPane.ERROR_MESSAGE);
281         return;
282     }
283
284     i_2021 = 0; j_2021 =0 ;
285     stepCount_2021 = 1;
286     sorting_2021 = true;
287     stepButton_2021.setEnabled(true);
288     stepArea_2021.setText("");
289     panelArray_2021.removeAll();
290     labelArray_2021 = new JLabel[array_2021.length];
291     for (int k_2021 = 0 ; k_2021 < array_2021.length ; k_2021++) {
292         labelArray_2021[k_2021] = new JLabel(String.valueOf(array_2021[k_2021]));
293         labelArray_2021[k_2021].setFont(new Font("Arial", Font.BOLD, 24));
294         labelArray_2021[k_2021].setOpaque(true);
295         labelArray_2021[k_2021].setBackground(Color.WHITE);
296
297         labelArray_2021[k_2021].setBorder(BorderFactory.createLineBorder(Color.BLACK));
298         labelArray_2021[k_2021].setPreferredSize(new Dimension(50,50));
299         labelArray_2021[k_2021].setHorizontalAlignment(SwingConstants.CENTER);
300         panelArray_2021.add(labelArray_2021[k_2021]);
301     }
302     panelArray_2021.revalidate();
303     panelArray_2021.repaint();
304
305
306 }
307 private void performStep() {
308
309     if (!sorting_2021 || i_2021 >= array_2021.length - 1) {
310         sorting_2021 = false;
311         stepButton_2021.setEnabled(false);
312         JOptionPane.showMessageDialog(this, "Sorting selesai");
313         return;
314     }
315
316     resetHighlights();
317
318     StringBuilder stepLog = new StringBuilder();
319
320     labelArray_2021[j_2021].setBackground(Color.CYAN);
321     labelArray_2021[j_2021 + 1].setBackground(Color.CYAN);
322
323     int nilaiKiri_2021 = array_2021[j_2021];
324     int nilaiKanan_2021 = array_2021[j_2021 + 1];
325
326     if (nilaiKiri_2021 > nilaiKanan_2021) {
327
328         // Swap yang benar
329         int temp_2021 = array_2021[j_2021];
330         array_2021[j_2021] = array_2021[j_2021 + 1];
331         array_2021[j_2021 + 1] = temp_2021;
332
333         stepLog.append("Langkah ")
334             .append(stepCount_2021)
335             .append(": Menukar ")
336             .append(nilaiKiri_2021)
337             .append(" dengan ")
338             .append(nilaiKanan_2021)
339             .append("\n");
340

```

```

341     } else {
342         stepLog.append("Langkah ")
343             .append(stepCount_2021)
344             .append(": Tidak ada pertukaran antara ")
345             .append(nilaiKiri_2021)
346             .append(" dan ")
347             .append(nilaiKanan_2021)
348             .append("\n");
349     }
350 }
351
352 stepLog.append("Hasil: ")
353     .append(arrayToString(array_2021))
354     .append("\n\n");
355
356 stepArea_2021.append(stepLog.toString());
357
358 updateLabels();
359
360 j_2021++;
361
362 if (j_2021 >= array_2021.length - i_2021 - 1) {
363     j_2021 = 0;
364     i_2021++;
365 }
366
367 stepCount_2021++;
368
369 if (i_2021 >= array_2021.length - 1) {
370     sorting_2021 = false;
371     stepButton_2021.setEnabled(false);
372     JOptionPane.showMessageDialog(this, "Sorting Selesai");
373 }
374 }
375 private void updateLabels() {
376     for (int k = 0 ; k < array_2021.length; k++) {
377         labelArray_2021[k].setText(String.valueOf(array_2021[k]));
378     }
379 }
380
381 private void reset() {
382     inputField_2021.setText("");
383     panelArray_2021.removeAll();
384     panelArray_2021.revalidate();
385     panelArray_2021.repaint();
386     stepArea_2021.setText("");
387     stepButton_2021.setEnabled(false);
388     sorting_2021 = false;
389     i_2021 = 1;
390     j_2021 = 1;
391     stepCount_2021 = 1;
392 }
393
394 private void resetHighlights() {
395     for (JLabel label : labelArray_2021) {
396         label.setBackground(Color.WHITE);
397     }
398 }
399
400 private String arrayToString(int[] arr) {
401     StringBuilder sb_2021 = new StringBuilder();
402     for (int k = 0; k < arr.length; k++) {
403         sb_2021.append(arr[k]);
404         if (k < arr.length - 1) sb_2021.append(", ");
405     }
406     return sb_2021.toString();
407 }
408
409 public static void main(String[] args) {
410     SwingUtilities.invokeLater(() -> {
411         BubbleSortGUI_2511532021 gui = new BubbleSortGUI_2511532021();
412         gui.setVisible(true);
413     });
414 }

```

```

415     }
416 }

```

Program ini adalah GUI Bubble Sort menggunakan Java Swing yang menampilkan proses sorting langkah per langkah secara visual.

Komponen UI: input field untuk memasukkan angka dipisah koma, panel array yang menampilkan tiap elemen sebagai kotak label berwarna, tombol "Langkah Selanjutnya" dan "Reset", serta text area di kanan yang mencatat log setiap langkah.

Alur kerja: setelah user memasukkan angka dan klik "Set Array", `setArrayFromInput()` mem-parse input dan membuat label kotak untuk tiap elemen. Setiap klik "Langkah Selanjutnya" memanggil `performStep()` yang membandingkan dua elemen bersebelahan (`j` dan `j+1`), mewarnai keduanya cyan, menukar jika kiri lebih besar, lalu mencatat hasilnya ke log. Variabel `i_2021` melacak pass dan `j_2021` melacak posisi perbandingan — ketika `j` mencapai ujung pass, `i` naik dan `j` reset ke 0, hingga `i >= length - 1` dan sorting dinyatakan selesai.

1.1 Merge Sort GUI

```

417 package pekan8_2511532021;
418
419 import java.awt.BorderLayout;
420 import java.awt.Color;
421 import java.awt.Dimension;
422 import java.awt.FlowLayout;
423 import java.awt.Font;
424
425 import java.util.LinkedList;
426 import java.util.Queue;
427
428 import javax.swing.BorderFactory;
429 import javax.swing.JButton;
430 import javax.swing.JFrame;
431 import javax.swing.JLabel;
432 import javax.swing.JOptionPane;
433 import javax.swing.JPanel;
434 import javax.swing.JScrollPane;
435 import javax.swing.JTextArea;
436 import javax.swing.JTextField;
437 import javax.swing.SwingConstants;
438 import javax.swing.SwingUtilities;
439
440 public class MergeSortGUI_2511532021 extends JFrame {
441
442     private static final long serialVersionUID_2021 = 1L;
443
444     private int[] array_2021;
445     private JLabel[] labelArray_2021;
446     private JButton stepButton_2021, resetButton_2021, setButton_2021;
447     private JTextField inputField_2021;
448     private JPanel panelArray_2021;
449     private JTextArea stepArea_2021;
450
451     private int i_2021, j_2021, k_2021;
452     private int left_2021, mid_2021, right_2021;
453     private int[] temp_2021;

```

```

454
455     private boolean isMerging_2021 = false;
456     private boolean copying_2021 = false;
457     private int stepCount_2021 = 1;
458
459     private Queue<int[]> mergeQueue_2021 = new LinkedList<>();
460
461     public MergeSortGUI_2511532021() {
462         setTitle("Merge Sort Langkah per Langkah");
463         setSize(750, 400);
464         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
465         setLocationRelativeTo(null);
466         setLayout(new BorderLayout());
467
468         JPanel inputPanel_2021 = new JPanel(new FlowLayout());
469         inputField_2021 = new JTextField(30);
470         setButton_2021 = new JButton("Set Array");
471         inputPanel_2021.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
472         inputPanel_2021.add(inputField_2021);
473         inputPanel_2021.add(setButton_2021);
474
475         panelArray_2021 = new JPanel();
476         panelArray_2021.setLayout(new FlowLayout());
477
478         JPanel controlPanel_2021 = new JPanel();
479         stepButton_2021 = new JButton("Langkah Selanjutnya");
480         resetButton_2021 = new JButton("Reset");
481         stepButton_2021.setEnabled(false);
482         controlPanel_2021.add(stepButton_2021);
483         controlPanel_2021.add(resetButton_2021);
484
485         stepArea_2021 = new JTextArea(8, 60);
486         stepArea_2021.setEditable(false);
487         stepArea_2021.setFont(new Font("Monospaced", Font.PLAIN, 14));
488         JScrollPane scrollPane_2021 = new JScrollPane(stepArea_2021);
489
490         add(inputPanel_2021, BorderLayout.NORTH);
491         add(panelArray_2021, BorderLayout.CENTER);
492         add(controlPanel_2021, BorderLayout.SOUTH);
493         add(scrollPane_2021, BorderLayout.EAST);
494
495         setButton_2021.addActionListener(e_2021 -> setArrayFromInput_2021());
496         stepButton_2021.addActionListener(e_2021 -> performStep_2021());
497         resetButton_2021.addActionListener(e_2021 -> reset_2021());
498     }
499
500     private void setArrayFromInput_2021() {
501         String text_2021 = inputField_2021.getText().trim();
502         if (text_2021.isEmpty()) return;
503
504         String[] parts_2021 = text_2021.split(",");
505         array_2021 = new int[parts_2021.length];
506
507         try {
508             for (int i_2021 = 0; i_2021 < parts_2021.length; i_2021++) {
509                 array_2021[i_2021] = Integer.parseInt(parts_2021[i_2021].trim());
510             }
511         } catch (NumberFormatException e_2021) {
512             JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
513                 "Error", JOptionPane.ERROR_MESSAGE);
514             return;
515         }
516
517         labelArray_2021 = new JLabel[array_2021.length];
518         panelArray_2021.removeAll();
519
520         for (int i_2021 = 0; i_2021 < array_2021.length; i_2021++) {
521             labelArray_2021[i_2021] = new JLabel(String.valueOf(array_2021[i_2021]));
522             labelArray_2021[i_2021].setFont(new Font("Arial", Font.BOLD, 24));
523             labelArray_2021[i_2021].setOpaque(true);
524             labelArray_2021[i_2021].setBackground(Color.WHITE);
525             labelArray_2021[i_2021].setBorder(BorderFactory.createLineBorder(Color.BLACK));
526             labelArray_2021[i_2021].setPreferredSize(new Dimension(50, 50));
527             labelArray_2021[i_2021].setHorizontalAlignment(SwingConstants.CENTER);

```

```

528         panelArray_2021.add(labelArray_2021[i_2021]);
529     }
530
531     mergeQueue_2021.clear();
532     generateMergeSteps_2021(0, array_2021.length - 1);
533
534     stepButton_2021.setEnabled(true);
535     stepArea_2021.setText("");
536     stepCount_2021 = 1;
537     isMerging_2021 = false;
538     copying_2021 = false;
539
540     panelArray_2021.revalidate();
541     panelArray_2021.repaint();
542 }
543
544 private void generateMergeSteps_2021(int left_2021, int right_2021) {
545     if (left_2021 < right_2021) {
546         int mid_2021 = left_2021 + (right_2021 - left_2021) / 2;
547
548         generateMergeSteps_2021(left_2021, mid_2021);
549         generateMergeSteps_2021(mid_2021 + 1, right_2021);
550
551         mergeQueue_2021.add(new int[] { left_2021, mid_2021, right_2021 });
552     }
553 }
554
555 private void performStep_2021() {
556     resetHighlights_2021();
557
558     if (!isMerging_2021 && !mergeQueue_2021.isEmpty()) {
559         int[] range_2021 = mergeQueue_2021.poll();
560
561         left_2021 = range_2021[0];
562         mid_2021 = range_2021[1];
563         right_2021 = range_2021[2];
564
565         temp_2021 = new int[right_2021 - left_2021 + 1];
566
567         i_2021 = left_2021;
568         j_2021 = mid_2021 + 1;
569         k_2021 = 0;
570
571         copying_2021 = false;
572         isMerging_2021 = true;
573
574         stepArea_2021.append(
575             "Langkah " + stepCount_2021++
576             + ": Mulai merge dari "
577             + left_2021 + " ke "
578             + right_2021 + "\n"
579         );
580
581         return;
582     }
583
584     if (isMerging_2021 && !copying_2021) {
585         if (i_2021 <= mid_2021 && j_2021 <= right_2021) {
586             labelArray_2021[i_2021].setBackground(Color.CYAN);
587             labelArray_2021[j_2021].setBackground(Color.CYAN);
588
589             if (array_2021[i_2021] <= array_2021[j_2021]) {
590                 temp_2021[k_2021++] = array_2021[i_2021++];
591             } else {
592                 temp_2021[k_2021++] = array_2021[j_2021++];
593             }
594
595             stepArea_2021.append(
596                 "Langkah " + stepCount_2021++
597                 + ": Bandingkan dan salin elemen\n"
598             );
599
600             return;
601

```

```

602         } else if (i_2021 <= mid_2021) {
603             temp_2021[k_2021++] = array_2021[i_2021++];
604
605             stepArea_2021.append(
606                 "Langkah " + stepCount_2021++
607                 + ": Salin sisa kiri\n"
608             );
609
610             return;
611
612         } else if (j_2021 <= right_2021) {
613             temp_2021[k_2021++] = array_2021[j_2021++];
614
615             stepArea_2021.append(
616                 "Langkah " + stepCount_2021++
617                 + ": Salin sisa kanan\n"
618             );
619
620             return;
621
622         } else {
623             copying_2021 = true;
624             k_2021 = 0;
625             return;
626         }
627     }
628
629     if (copying_2021 && k_2021 < temp_2021.length) {
630         array_2021[left_2021 + k_2021] = temp_2021[k_2021];
631         labelArray_2021[left_2021 + k_2021].setText(String.valueOf(temp_2021[k_2021]));
632         labelArray_2021[left_2021 + k_2021].setBackground(Color.GREEN);
633         k_2021++;
634
635         stepArea_2021.append("Langkah " + stepCount_2021++ + ": Tempelkan ke array
utama\n");
636         return;
637     }
638
639     if (copying_2021 && k_2021 == temp_2021.length) {
640         isMerging_2021 = false;
641         copying_2021 = false;
642     }
643
644     if (mergeQueue_2021.isEmpty() && !isMerging_2021) {
645         stepArea_2021.append("Selesai.\n");
646         stepButton_2021.setEnabled(false);
647         JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
648     }
649 }
650
651 private void resetHighlights_2021() {
652     if (labelArray_2021 == null) return;
653
654     for (JLabel label_2021 : labelArray_2021) {
655         label_2021.setBackground(Color.WHITE);
656     }
657 }
658
659 private void reset_2021() {
660     inputField_2021.setText("");
661     panelArray_2021.removeAll();
662     panelArray_2021.revalidate();
663     panelArray_2021.repaint();
664     stepArea_2021.setText("");
665     stepButton_2021.setEnabled(false);
666     mergeQueue_2021.clear();
667     isMerging_2021 = false;
668     copying_2021 = false;
669     stepCount_2021 = 1;
670 }
671
672 public static void main(String[] args) {
673     SwingUtilities.invokeLater(() -> {
674         MergeSortGUI_2511532021 frame_2021 = new MergeSortGUI_2511532021();

```

```
675         frame_2021.setVisible(true);  
676     });  
677 }  
678 }
```

Program ini adalah GUI Merge Sort langkah per langkah menggunakan Java Swing, dengan komponen UI yang sama seperti BubbleSortGUI — input field untuk memasukkan angka dipisah koma, panel kotak label untuk menampilkan array, tombol "Langkah Selanjutnya" dan "Reset", serta text area di kanan untuk mencatat log setiap langkah.

Yang membedakan program ini secara arsitektur adalah penggunaan `Queue<int[]> mergeQueue` — ketika user menekan "Set Array", method `generateMergeSteps_2021()` langsung menjalankan rekursi penuh dan menyimpan semua operasi merge yang perlu dilakukan ke dalam queue sebagai `{left, mid, right}`. Artinya seluruh struktur rekursi diselesaikan di awal, dan tombol langkah selanjutnya tinggal mengeksekusi antrian satu per satu tanpa perlu rekursi saat runtime.

`performStep_2021()` bekerja dalam tiga fase yang dikendalikan oleh dua flag boolean `isMerging` dan `copying` — fase pertama mengambil range dari queue dan menyiapkan array sementara temp, fase kedua membandingkan elemen kiri dan kanan lalu menyalin yang lebih kecil ke temp dengan highlight cyan, dan fase ketiga menempelkan isi temp kembali ke array utama satu per satu dengan highlight hijau — sehingga setiap klik tombol hanya mengeksekusi satu sub-langkah dan user dapat mengamati proses merge secara sangat detail.

BAB 3 TUGAS

1.1 Program Lagu

```
679 package pekan8_2511532021;
680
681 public class Lagu_2511532021 {
682     String judul ;
683     String penyanyi;
684     int durasi;
685     public Lagu_2511532021(String judul, String penyanyi, int durasi) {
686         this.judul = judul ;
687         this.durasi = durasi ;
688         this.penyanyi = penyanyi;
689     }
690
691     public String getJudul() {
692         return this.judul;
693     }
694
695     public String getPenyanyi() {
696         return this.penyanyi;
697     }
698
699     public int getDurasi() {
700         return this.durasi;
701     }
702
703     public void setJudul(String judul) {
704         this.judul = judul;
705     }
706
707
708     public void setPenyanyi(String penyanyi) {
709         this.penyanyi = penyanyi;
710     }
711
712     public void setDurasi(int durasi) {
713         this.durasi = durasi;
714     }
715
716 }
```

Ini adalah class model Lagu_2511532021 yang merepresentasikan sebuah lagu dengan tiga atribut: judul, penyanyi, dan durasi (dalam detik). Class ini mengikuti pola standar Java Bean — konstruktor untuk inisialisasi ketiga atribut sekaligus, getter untuk membaca nilai, dan setter untuk mengubah nilai masing-masing atribut. Class ini digunakan sebagai tipe data dalam SortingLagu_2511532021 untuk menyimpan dan mengurutkan data lagu.

3.2 Program Sorting Lagu

```
4 package pekan8_2511532021;
5 import java.util.*;
6
7 public class SortingLagu_2511532021 {
8     static int index_2021 = 0;
9     static Lagu_2511532021[] arraylagu_2021 = new Lagu_2511532021[20];
10    static Scanner input_2021 = new Scanner(System.in);
11    public static void InputLagu(int index_2021) {
12        System.out.print("masukkan judul : ");
13        String judul_2021 = input_2021.nextLine();
14        System.out.print("masukkan penyanyi : ");
15        String Penyanyi_2021 = input_2021.nextLine();
16        System.out.print("masukkan durasi (detik) : ");
17        int durasi_2021 = input_2021.nextInt();
18        input_2021.nextLine();
19        Lagu_2511532021 newLagu = new Lagu_2511532021(judul_2021, Penyanyi_2021,
durasi_2021);
20        arraylagu_2021[index_2021] = newLagu;
21
22    }
23
24
25    public static void printPlaylist() {
26        for (int i_2021 = 0 ; i_2021 < index_2021; i_2021++) {
27            System.out.println("Judul lagu :" + arraylagu_2021[i_2021].getJudul());
28            System.out.println("Penyanyi lagu :" + arraylagu_2021[i_2021].getPenyanyi());
29            System.out.println("Durasi (detik) :" + arraylagu_2021[i_2021].getDurasi() +
"\n");
30        }
31
32        System.out.println();
33    }
34
35
36    static void swap_2021(int[] arr_2021, int i_2021, int j_2021) {
37        int temp_2021 = arr_2021[i_2021];
38        arr_2021[i_2021] = arr_2021[j_2021];
39        arr_2021[j_2021] = temp_2021;
40    }
41
42
43    //quick sort
44    static int partition_2021(Lagu_2511532021[] arr_2021, int low_2021, int high_2021) {
45        medianOfThree_2021(arr_2021, low_2021, high_2021);
46        int pivot_2021 = arr_2021[high_2021].getDurasi();
47        int i_2021 = low_2021 - 1;
48
49        for (int j_2021 = low_2021; j_2021 <= high_2021 - 1; j_2021++) {
50            if (arr_2021[j_2021].getDurasi() < pivot_2021) {
51                i_2021++;
52                swap_2021(arr_2021, i_2021, j_2021);
53            }
54        }
55        swap_2021(arr_2021, i_2021 + 1, high_2021);
56        return i_2021 + 1;
57    }
58
59    static void swap_2021(Lagu_2511532021[] arr_2021, int i_2021, int j_2021) {
60        Lagu_2511532021 temp_2021 = arr_2021[i_2021];
61        arr_2021[i_2021] = arr_2021[j_2021];
62        arr_2021[j_2021] = temp_2021;
63    }
64
65    static void medianOfThree_2021(Lagu_2511532021[] arr_2021, int low_2021, int
high_2021) {
66        int mid_2021 = low_2021 + (high_2021 - low_2021) / 2;
67
68        if (arr_2021[low_2021].getDurasi() > arr_2021[mid_2021].getDurasi())
69            swap_2021(arr_2021, low_2021, mid_2021);
70
71        if (arr_2021[low_2021].getDurasi() > arr_2021[high_2021].getDurasi())
72            swap_2021(arr_2021, low_2021, high_2021);
```

```

73         if (arr_2021[mid_2021].getDurasi() > arr_2021[high_2021].getDurasi())
74             swap_2021(arr_2021, mid_2021, high_2021);
75
76         swap_2021(arr_2021, mid_2021, high_2021);
77     }
78
79
80     static void quickSort_2021(Lagu_2511532021[] arr_2021, int low_2021, int high_2021) {
81         if (low_2021 < high_2021) {
82             int pi_2021 = partition_2021(arr_2021, low_2021, high_2021);
83             quickSort_2021(arr_2021, low_2021, pi_2021 - 1);
84             quickSort_2021(arr_2021, pi_2021 + 1, high_2021);
85         }
86     }
87     //end of quick sort blablablablabl
88
89     public static void main(String[] args) {
90
91         while (true) {
92             System.out.println("Pilih opsi");
93             System.out.println("1. Tambah Lagu");
94             System.out.println("2. Tampilkan informasi playlist sekarang");
95             System.out.println("3. Sort Playlist berdasarkan durasi");
96             System.out.println("4. Keluar");
97             System.out.print("Pilihan mu : ");
98             int pilihan_2021 = input_2021.nextInt();
99             input_2021.nextLine();
100             if (pilihan_2021 == 1) {
101                 InputLagu(index_2021++);
102
103             }
104             else if (pilihan_2021 == 2) {
105                 if (index_2021 == 0) {
106                     System.out.println("PLAYLIST KOSONG");
107                     continue ;
108                 }
109                 printPlaylist();
110             }
111
112             else if (pilihan_2021 == 3) {
113                 if (index_2021 == 0) {
114                     System.out.println("PLAYLIST KOSONG");
115                     continue;
116                 }
117                 quickSort_2021(arrayLagu_2021, 0, index_2021 - 1);
118                 printPlaylist();
119             }
120
121             else if (pilihan_2021 == 4) {
122                 return;
123             }
124
125             else {
126                 System.out.println("Ndak da pilihannyo buyan");
127             }
128         }
129     }
130 }

```

Program ini menggunakan algoritma sorting quick sort karena algoritma tersebut quick (apalagi memang alasannya?).

Program ini adalah aplikasi playlist berbasis konsol yang memungkinkan user menambah, menampilkan, dan mengurutkan lagu berdasarkan durasi.

Data lagu disimpan dalam static array `arraylagu_2021` bertipe `Lagu_2511532021` berkapasitas 20, dengan `index_2021` sebagai counter pengisian. Method `InputLagu()` menerima input judul, penyanyi, dan durasi dari user lalu menyimpannya sebagai objek `Lagu_2511532021` ke array, sementara `printPlaylist()` mencetak semua lagu yang sudah ditambahkan dari index 0 hingga `index_2021`.

Untuk sorting, program menggunakan Quick Sort dengan median-of-three pivot yang bekerja langsung pada array objek `Lagu_2511532021` — `medianOfThree_2021()` memilih pivot terbaik dari elemen low, mid, dan high berdasarkan nilai durasi, lalu `partition_2021()` mempartisi array sehingga lagu dengan durasi lebih pendek berada di kiri dan lebih panjang di kanan, dan `quickSort_2021()` memanggil keduanya secara rekursif. Perlu diperhatikan ada method `swap_2021()` yang di-overload — satu untuk `int[]` dan satu untuk `Lagu_2511532021[]`, namun versi `int[]` tidak lagi digunakan karena sorting sudah bekerja langsung pada array objek.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Keseluruhan program dalam package pekan8_2511532021 ini merupakan implementasi berbagai algoritma sorting sebagai bahan pembelajaran struktur data dan algoritma. Terdapat tiga algoritma utama yang diimplementasikan yaitu Merge Sort, Quick Sort, dan Shell Sort — masing-masing dalam versi standalone dengan main() sendiri untuk pengujian, sekaligus Quick Sort juga diintegrasikan ke dalam aplikasi playlist SortingLagu_2511532021 yang bekerja pada array objek.

Dari sisi struktur kode, setiap program mengikuti pola yang konsisten: array diinisialisasi dengan data hardcoded, dicetak sebelum sorting, diproses, lalu dicetak kembali sesudahnya untuk memperlihatkan perubahan. Quick Sort menggunakan teknik median-of-three untuk pemilihan pivot yang lebih optimal, Merge Sort menggunakan array sementara untuk proses penggabungan, dan Shell Sort menggunakan gap yang terus dihalving sebagai pengembangan dari insertion sort.

Dua program GUI yaitu BubbleSortGUI_2511532021 dan MergeSortGUI_2511532021 menambahkan dimensi visualisasi menggunakan Java Swing, memungkinkan user melihat proses sorting langkah per langkah dengan highlight warna pada elemen yang sedang dibandingkan atau dipindahkan. Pendekatan keduanya berbeda — BubbleSortGUI menghitung langkah secara on-the-fly setiap klik, sedangkan MergeSortGUI lebih canggih dengan meng-generate seluruh antrian langkah di awal menggunakan Queue sebelum eksekusi dimulai.

Secara keseluruhan, package ini mencakup spektrum yang cukup lengkap dari pembelajaran sorting — mulai dari implementasi algoritma murni, aplikasi nyata pada data objek melalui playlist lagu, hingga visualisasi interaktif berbasis GUI. Class Lagu_2511532021 sebagai model data juga menunjukkan penerapan konsep OOP dasar seperti encapsulation dengan getter dan setter, yang kemudian dimanfaatkan oleh SortingLagu_2511532021 untuk mengurutkan objek berdasarkan atribut tertentu yaitu durasi.

